

Reviewer Pack — Minimal Order of p-Adic K-theory and Communication Complexit...

Entry 0439abc842d6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 03:18:00 UTC

Minimal Order of p-Adic K-theory and Communication Complexity Rank Inequality

Entry ID: 0439abc842d6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 03:18:00 UTC

1. Conjecture

Field A (mathematical branch): Algebra (p-Adic K-theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -variables communication complexity problem with matrix rank r , the minimal order of a p-adic K-theoretic invariant $\kappa(\pi)$ for the associated Tseitin formula π is upper bounded by r^2 , i.e., $\kappa(\pi) \leq O(r^2)$.

Rationale (proposer's reasoning):

p-Adic K-theory provides a framework to study arithmetic properties in number theory and algebraic geometry, which may offer new insights into the computational complexity of communication problems. If the rank of the matrix associated with a communication problem is high, then a higher order p-adic K-theoretic invariant should correspond to more complex communication tasks.

Taxonomy category: Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c23e6c283d2507e0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman's rank correlation coefficient between $\kappa(\pi)$ and r^2 is statistically significant ($p\text{-value} \leq 0.05$), with all seeds showing $\kappa(\pi) \leq O(r^2)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for i in range(1, n+1):
            clause = [f'¬{variables[i-1]}', f'{variables[n+i-1]}']
            clauses.append(clause)
            clause = [f'{variables[i-1]}', f'¬{variables[n+i-1]}']
            clauses.append(clause)
        for i in range(1, n+1):
            for j in range(i+1, n+1):
                clause = [f'¬{variables[i-1]}', f'¬{variables[j-1]}', f'{variables[2*n+i+j-2]}']
                clauses.append(clause)
                clause = [f'{variables[i-1]}', f'{variables[j-1]}', f'¬{variables[2*n+i+j-2]}']
                clauses.append(clause)
        return variables, clauses

    def matrix_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
        for i in range(n):
            if all(matrix[j][i] == 0 for j in range(m)):
                continue
            pivot_row = next(j for j in range(i, m) if matrix[j][i] != 0)
            matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
            for j in range(m):
                if j != i:
                    factor = -matrix[j][i] / matrix[i][i]
                    for k in range(n):
                        matrix[j][k] += factor * matrix[i][k]
            rank += 1
```

```

    return rank

def p_adic_k_theory_invariant(matrix):
    m, n = len(matrix), len(matrix[0])
    invariant = 0
    for i in range(m):
        for j in range(n):
            if matrix[i][j] != 0:
                invariant += abs(Fraction(matrix[i][j]).numerator) * abs(Fraction(matrix[i][j]))
    return invariant

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    variables, clauses = generate_tseitin_formula(n)
    matrix = [[0] * (2*n) for _ in range(2*n)]
    for clause in clauses:
        for var in clause:
            if var.startswith('-'):
                i = int(var[1:]) - 1
                matrix[i][n+i] = 1
            else:
                i = int(var) - 1
                matrix[n+i][i] = 1

    r = matrix_rank(matrix)
    kappa = p_adic_k_theory_invariant(matrix)

    results.append({
        "n": n,
        "r": r,
        "kappa": kappa,
        "conjecture_holds": kappa <= r**2
    })

metric_value = sum(result["kappa"] for result in results) / len(results)
instances_tested = len(results)
n_max = max(result["n"] for result in results)
conjecture_holds_all = all(result["conjecture_holds"] for result in results)
counterexample = "" if conjecture_holds_all else "mapping_undefined"

return {
    "metric_name": "p-adic K-theory invariant",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds_all,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 7 for i in range(5, 30)]

    results = []

```

```

for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_306eb44c.py", line 108, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_306eb44c.py", line 66, in run_trial
    variables, clauses = generate_tseitin_formula(n)
                          ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_306eb44c.py", line 25, in generate_tseitin_formula
    clause = [f'~{variables[i-1]}' if i % 2 == 1 else f'{variables[i-1]}' for i in range(1, n+1)]
              ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that it was unable to complete the computation and verify the conjecture. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13472
2	preregistration	ollama_remote	glm4:latest	0	0	9425
3	novelty	ollama_remote	glm4:latest	0	0	8752
4	novelty	ollama_remote	glm4:latest	0	0	8843
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17944
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36039
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16605
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14178
9	judge	ollama_remote	glm4:latest	0	0	53615

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 178872 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0439abc842d6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0439abc842d6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0439abc842d6.tar.gz` (if generated)